

7 - Tuples, dictionaries and sets

```
result = {}
for i in range(5):
    result[i] = i**2
```

Example 6 - words mapped to their lengths, only for words longer than 3:

```
words = ["apple", "cat", "banana", "dog", "elephant", "ox"]
result = {}
for word in words:
    if len(word) > 3:
        result[word] = len(word)
```

Course Participants

ex_07_09_course_participants.py

A school runs the same course in two consecutive semesters. We have sets of participants for each semester and want to analyse overlap and changes between the two groups.

Use this data:

```
autumn = {
    "Alice Johnson", "Bob Smith", "Charlie Brown",
    "Diana Prince", "Edward Norton", "Fiona Green", "George
Harris",
}

spring = {
    "Bob Smith", "Charlie Brown", "Diana Prince",
    "Hannah White", "Ivan Black", "Fiona Green", "Julia
Roberts",
}
```

Write a program that finds and displays:

- All participants across both semesters (union)
- Participants who attended both semesters (intersection)
- Participants who dropped out (autumn but not spring)
- New participants (spring but not autumn)
- Participants who attended exactly one semester (symmetric difference)
- The count for each group

Example run:

All participants: 10

Alice Johnson, Bob Smith, Charlie Brown, Diana Prince,
Edward Norton, Fiona Green, George Harris, Hannah White,
Ivan Black, Julia Roberts

Attended both semesters: 4

Bob Smith, Charlie Brown, Diana Prince, Fiona Green

Dropped out (autumn only): 3

Alice Johnson, Edward Norton, George Harris

New participants (spring only): 3

Hannah White, Ivan Black, Julia Roberts

Exactly one semester: 6

Alice Johnson, Edward Norton, George Harris,
Hannah White, Ivan Black, Julia Roberts

8 Classes and objects - the basics

Learning objectives

- Understand what a class and an object are, and know the difference between a class and an instance
- Define a class with `__init__()`, instance attributes and instance methods
- Understand what `self` is and how Python passes it automatically
- Use dot notation to access attributes and call methods
- Assess when something should become a class and when a function is enough
- Understand Python's convention for "private" attributes with a leading underscore
- Implement `__str__()` for readable output of objects
- Design classes that collaborate - such as `Account` and `Transaction`
- Use properties with `@property` and `@name.setter` for access control and validation
- Understand what class variables and class methods are, and distinguish between `@classmethod` and `@staticmethod`
- Implement common dunder methods for operator overloading: `__eq__()`, `__lt__()`, `__gt__()`, `__getitem__()`, `__contains__()`
- Implement `__bool__()` to control the truthiness of custom objects

8.1 Introduction

In Python and other languages, *classes* are a way to create our own data types, combining data (attributes) and functionality (methods) in one structure. An *object* is an *instance* of a class - a concrete occurrence.

We have already worked with built-in classes such as `int`, `str` and `list`. These were created by the Python developers in the same way as we will now create our own.

`int` is a relatively simple class that can store an integer and perform operations on it.

`list` is far more advanced - it can store a collection of elements, sort them, iterate over them and much more.

We create objects of built-in types like this:

```
an_int = 1          # Python infers that this is int from
context
an_int = int(1)     # Explicit use of the constructor
```

For our own classes we must always use the constructor (explained below) explicitly - Python cannot infer the type from context the way it does for `int` and `str`.

We start with simple, conceptual examples and gradually build up to more advanced classes.

8.2 Creating a class

A class is a template or blueprint for creating objects. We define what data (attributes) and what behaviour (methods) the objects will have. Each object created from the class gets its own values for the attributes.

```
1: # file: sc_08_01_Car.py
2: class Car:
3:     def __init__(self, brand, color):
4:         self._brand = brand
5:         self._color = color
6:
7:     def drive(self):
8:         print(f"{self._brand} car is driving!")
9:
10:    def show_info(self):
11:        print(f"Car: {self._brand}, Color: {self._color}")
12:
13: my_car = Car("Toyota", "Blue")
14: another_car = Car("Volvo", "Red")
15:
16: my_car.drive()
17: my_car.show_info()
18: another_car.show_info()
```

Output:

```
Toyota car is driving!
Car: Toyota, Color: Blue
Car: Volvo, Color: Red
```

8 - Classes and objects - the basics

Code comments:

Line 2

The class name starts with a capital letter - this is a Python convention. When multiple words are used, each word is capitalised (PascalCase), for example `BankAccount` or `CarFactory`.

Line 3-5

`__init__()` is the constructor - it is called automatically when we create an object (lines 13-14). The parameter `self` is a reference to the object itself. The attributes `_brand` and `_color` are created on the object with `self._name`. *A single underscore signals that they are intended for internal use.*

Line 7-11

The instance methods `drive()` and `show_info()` take `self` as the first parameter so they have access to the object's own attributes.

Line 13-14

We create two `Car` objects. The arguments in parentheses are passed to `__init__()` as `brand` and `color`. Python inserts `self` automatically.

What exactly is the `Car` class that we have created?

We have simply tried to model something *that exists in the real world* - a car. Admittedly, our first attempt may seem rather silly, and it obviously cannot be used for anything useful in this version.

We chose `Car` as the first example because it is something recognizable. It also lets us demonstrate the difference between what a `Car` *can do* - its methods, such as `drive()` - and what *properties* it has - its attributes, such as `_brand` and `_color`.

Not all classes that we will create will be as concrete as `Car`. Often, we need classes that are more abstract, designed to collect functionality and properties into one unit.

8.3 `__init__()` - the constructor

`__init__()` is called automatically when an object is created. It initialises the object's attributes and ensures the object is in a valid state from the very start.

Attributes should always be created in `__init__()`, not scattered around other methods. This gives a clear overview and predictability - and avoids

bugs where a method tries to use an attribute that has not been set yet. Attributes are created with `self._name` and live as long as the object exists.

In Python, this is primarily a style rule, not a technical requirement. An attribute created in another method is just as much an instance attribute as an attribute created in `__init__()`. The important point is that the attribute is created through `self`.

This is a major difference from languages such as C++ and Java, where fields are usually declared explicitly in the class definition before they are used. In Python, an object's attributes can appear while the program is running. This gives flexibility, but also more responsibility: if attributes are created in different methods, it becomes harder to see which state the object may have. Therefore, we should always create the object's fixed attributes in `__init__()`.

8.4 What is self?

To understand `self` it is important to distinguish between two things:

- The *code* defined in a class is *shared by all objects*.
- The *attributes* - the data - are *unique to each object*.

When we write `my_car.drive()` Python rewrites it as `Car.drive(my_car)` behind the scenes. That is how `self` gets its value; - Python passes a reference to the current object as the first argument. The same happens at creation:

```
my_car = Car("Toyota", "Blue")
# Becomes:
Car.__init__(my_car, "Toyota", "Blue")
```

But wait - how can we pass `my_car` in when the object has not been created yet? The answer is that Python does this in two steps: first the method `__new__()` is called, which allocates memory and creates the object itself, then `__init__()` is called to initialise it. By the time `__init__()` is called, the object already exists, and `self` points to it.

8.5 Dot notation: .

The dot notation tells Python that we want to use something that belongs to an object - either a value (attribute) or a function (method). Without the dot Python does not know which object we mean.

When we write `my_car.drive()` we are asking the object `my_car` to run the code for `drive()`, applied to its own attributes.

8.6 When should something become a class?

When you model something that has both data and behaviour that naturally belong together, it is probably a good candidate for a class. The right question to ask is: does what I am trying to model have both data and behaviour that go together?

Create a class when:

- You generally need to model something in the real world (at least, this would be a *candidate* to become a class)
- You have several related pieces of data that belong together
- You need functions that operate on this data
- You need to create multiple instances with the same structure but different content
- You want to organise the code in a clean and reusable way

Not everything needs to become a class. A simple function that does one thing is often better than a class:

```
def calculate_area(radius):  
    return 3.14 * radius ** 2
```

There is no need for a class here - the function does one thing and contains no data.

UML notation

As we now begin to build classes, it can be useful to have a visual tool for communicating to others how we think of a class, including its methods and attributes. UML - Unified Modeling Language - is a useful tool for this. We will return to it in the chapter on advanced classes, Chapter 10.

8.7 A more advanced class: Account

We will now build an Account class representing a bank account, and extend it step by step. Each step introduces new concepts:

- Basic class with methods `deposit()`, `withdraw()` etc
- A separate Transaction class to store transactions
- Properties for access control
- Class variables and class methods

8.7.1 Basic Account class

```
1: # file: sc_08_02_Account1.py
2: class Account:
3:     def __init__(self, cust_id, account_no, start_balance,
interest):
4:         self._cust_id    = cust_id
5:         self._account_no = account_no
6:         self._balance    = start_balance
7:         self._interest   = interest
8:
9:     def get_balance(self):
10:         return self._balance
11:
12:     def set_balance(self, new_balance):
13:         self._balance = new_balance
14:
15:     def deposit(self, amount):
16:         if amount > 0:
17:             self._balance += amount
18:         return self._balance
19:
20:     def withdraw(self, amount):
21:         if amount <= self._balance:
22:             self._balance -= amount
23:         return self._balance
24:
25:     def add_monthly_interest(self):
26:         self._balance += self.calculate_monthly_interest()
27:
28:     def calculate_monthly_interest(self):
29:         return self._balance * self._interest / 100 / 12
30:
31:     def __str__(self):
32:         return (f"Customer id    = {self._cust_id}\n"
33:               f"Account no     = {self._account_no}\n"
34:               f"Balance       = {self._balance:.2f}\n"
35:               f"Interest      = {self._interest}%")
```

Code comments:

Line 3-7

`__init__()` receives customer ID, account number, starting balance and rate, and stores them as attributes. Note that interest is given as an integer - a 5% rate is stored as 5. All attributes have a leading underscore, signalling that client code should not change them directly.

8 - Classes and objects - the basics

Line 9-13

`get_balance()` and `set_balance()` are traditional getter/setter methods. We will replace them with properties in the next version.

Line 15-23

`deposit()` and `withdraw()` perform simple validations: deposits must be positive, withdrawals cannot exceed the balance. Both return the new balance.

Line 28-29

We divide by 100 to convert from a percentage to a decimal, and by 12 to get the monthly interest rate.

Line 31-35

`__str__()` is called implicitly when we pass the object to `print()`. If the method is missing, Python uses `__str__()` from the ultimate base class, which just gives a memory address. Try commenting out the `__str__()` method here, and see what happens.

Client code:

```
1: # file: sc_08_03_use_Account1_.py
2: from sc_08_02_Account1 import Account
3:
4: account1 = Account(1, 1000, 50000, 7)
5: account2 = Account(2, 1001, 10000, 5)
6: print(account1)
7: print(account2)
8: Account.deposit(account1, 500)
9: print(f'New balance after deposit:{account1.get_balance()}')
10: account1.withdraw(1000)
11: print(f'New balance after withdr.:{account1.get_balance()}')
12: account1.add_monthly_interest()
13: print(f'New balance after monthly interest:
{account1._balance}')
```

Line 6-7

`print()` finds `__str__()` in `Account` and calls it implicitly.

Line 13

We access `_balance` directly -, even if we have marked the attribute as private - Python does not stop us. The underscore is just a convention saying "stay away (use `get_balance()` instead)".

Output from the client code:

```
Customer id = 1
Account no = 1000
Balance = 50000
Interest = 7

Customer id = 2
Account no = 1001
Balance = 10000
Interest = 5

New balance after deposit: 50500
New balance after withdr.: 49500
New balance after monthly interest: 49788.75
```

8.8 Python has no private data members

Programmers from C++ and Java are used to attributes being declared private so that client code cannot access them directly. Python *does not* have this mechanism - all attributes and methods are accessible from outside.

The convention in Python is that a single leading underscore signals "internal use - keep away". It is an agreement between developers, not a technical barrier. Python is built on trust between developers rather than enforced rules.

Discussion / dilemma: when should we use a leading underscore?

As mentioned - in C++ and Java there are clear rules: private means inaccessible from outside, public means open. Python does not have this distinction, so the convention is a little more nuanced:

Use a leading underscore (`_name`) when the attribute is an implementation detail - something client code should not deal with directly. This applies especially when you plan to add validation via properties, or when the value should not be freely changed from outside.

Omit the underscore when the attribute is meant to be directly accessible and you do not need to control access. This is perfectly Pythonic for simple data classes where openness is a deliberate design decision.

In practice, many Python developers choose to use underscore consistently on all instance attributes - it is a safe default that makes it easy to add properties later without changing the client code.

Under the hood: name mangling with double underscore

You can start an attribute name with a double underscore, e.g. `self.__balance`. Python then renames it internally to `__Account__balance` - this is called name mangling. The purpose is to avoid name conflicts in inheritance, not to create truly private attributes. The attribute is still accessible if you know the internal name. *Name mangling is not recommended as a replacement for the private mechanism from C++/Java.*

8.9 Account with transaction log

We introduce a Transaction class to log what happens on the account. This is a good example of classes collaborating - Account creates Transaction objects and stores them in a list.

```
# file: sc_08_04_Account2.py
from datetime import datetime

class Transaction:
    def __init__(self, amount, trans_type):
        self._amount = amount
        self._trans_type = trans_type # "deposit", "withdraw",
        "interest"
        self._timestamp = datetime.now()

    def __str__(self):
        return (f"{self._timestamp:%Y-%m-%d %H:%M:%S} | "
                f"{self._trans_type.capitalize():8} | "
                f"{self._amount:8.2f}")
```

Transaction is deliberately simple - it stores data and knows how to print itself. It is Account's responsibility to create and store transactions, that is, objects of this class (Transactions).

In Account we add a list for transactions and update the methods that change the balance. Here we show only the parts that have changed:

```
class Account:
    def __init__(self, cust_id, account_no, start_balance,
        interest):
        self._cust_id = cust_id
        self._account_no = account_no
        self._balance = start_balance
```

```
        self._interest      = interest
        self._transactions = []      # NEW

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            self._transactions.append(Transaction(amount,
"deposit")) # NEW
        return self._balance

    def withdraw(self, amount):
        if amount <= self._balance:
            self._balance -= amount
            self._transactions.append(Transaction(amount,
"withdraw")) # NEW
        return self._balance

    def add_monthly_interest(self):
        monthly_interest = self.calculate_monthly_interest()
        self._balance    += monthly_interest
        self._transactions.append(Transaction(monthly_interest,
"interest")) # NEW

    def print_transactions(self):
        print("Date and time          | Type      |   Amount")
        print("-" * 42)
        for trans in self._transactions:
            print(trans)
```

Note that `print(trans)` calls `Transaction.__str__()` implicitly - exactly the same as happens when we print an `Account` object.

Client code

```
# file: sc_08_05_use_Account2.py
from sc_08_04_Account2 import Account

account = Account(1, 1000, 50000, 7.0)

account.deposit(2000)
account.withdraw(500)
account.add_monthly_interest()

account.print_transactions()
print(account)
```

Output

Date and time	Type	Amount
2026-05-07 21:38:42	Deposit	2000.00
2026-05-07 21:38:42	Withdraw	500.00
2026-05-07 21:38:42	Interest	300.42

Customer id	=	1
Account no	=	1000
Balance	=	51800.42
Interest	=	7.0%

8.10 Properties for access control

Properties are the most Pythonic way to protect attributes. They allow us to control reading and writing, add validation, and make attributes read-only - without the client code needing to change.

With properties, client code accesses the attribute by its "clean" name - `account1.balance` - without dealing with the internal underscore representation. Inside the class we still use `self._balance`. Properties thus hide the implementation detail from the client code, as it should be.

The recipe for a property with validation:

1. The attribute follows the leading underscore convention:
`self._balance`
2. A getter method decorated with `@property` - called implicitly when the attribute name appears on the right side of "=" in client code
3. A setter method decorated with `@attribute_name.setter` - called implicitly when the attribute name appears on the left side of "=" in client code

Fact: what is a decorator?

`@property` and `@balance.setter` are examples of decorators - they start with `@` and are placed directly above a function or method definition. A decorator is a way to "wrap" or extend a function with extra behaviour, without changing the function itself.

In this case `@property` makes Python treat the method as an attribute rather than a regular method call - that is why we can write `account1.balance` instead of `account1.balance()`.

We will look more closely at how decorators work in a later chapter. For now it is enough to know that `@property` and `@name.setter` are the recipe for creating properties in Python.

```
# file: sc_08_06_Account3.py (extract, selected parts)

class Account:
    # ... __init__ & other methods unchanged

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, new_balance):
        if new_balance < 0:
            print("Balance cannot be negative!")
        else:
            self._balance = new_balance

    @property
    def interest(self):
        return self._interest

    @interest.setter
    def interest(self, new_interest):
        if new_interest < 0:
            print("Interest rate cannot be negative!")
        else:
            self._interest = new_interest
```

8 - Classes and objects - the basics

The client code can now look like this:

```
# file: sc_08_07_use_Account3.py
from sc_08_06_Account3 import Account

account1 = Account(123, 456, 1000, 2.5)
account1.deposit(500)
account1.deposit(1500)
account1.withdraw(200)
account1.withdraw(2000)account1.add_monthly_interest()
account1.print_transactions()

# Using properties
print(f"\nBalance after: {account1.balance:.2f}")
print(f"Interest rate: {account1.interest}%")

# Change balance and interest via properties
account1.balance = 2000
account1.interest = 3.0
print(f"\nNew balance: {account1.balance:.2f}")
```

The big gain is that the setter method can contain logic and validation. The client code does not see this - it just writes `account1.balance = value` regardless.

Output

Date and time	Type	Amount
2026-05-07 21:51:29	Deposit	500.00
2026-05-07 21:51:29	Deposit	1500.00
2026-05-07 21:51:29	Withdraw	200.00
2026-05-07 21:51:29	Withdraw	2000.00
2026-05-07 21:51:29	Interest	1.67

Balance after: 801.67
Interest rate: 2.5%

New balance: 2000.00

get_balance() or @property?

We replaced `get_balance()` with a property. When should you choose which?

Use a property when access should look like ordinary attribute reading - it gives more readable and Pythonic code. Use an explicit get method

when the operation is expensive (e.g. a database lookup) and you want the call to look like a function call, not a variable read.

Tip: when should you use properties?

- Use a property when you want to validate or restrict what can be set.
- Use a property when you want to hide internal implementation.
- Use a property for read-only attributes (getter only, no setter).
- Use a property for computed values derived from other attributes.
- Do not use a property for simple attributes without extra logic - plain attributes are simplest.

8.10.1 Another property example: Temperature

Let us look at another example where properties give a clear benefit. The class `Temperature` represents a temperature. We want the user to be able to read and set the temperature in both Celsius and Fahrenheit - and we want to guard against physically impossible values. Internally the class stores only one value: `_celsius`. Everything else is properties that compute from this.

Three properties with different roles:

- `celsius` - read and set the temperature in Celsius, with validation against absolute zero.
- `fahrenheit` - read and set the temperature in Fahrenheit. Internally the value is always stored as Celsius, so the conversion happens automatically.
- `kelvin` - read-only. Kelvin is uniquely determined by the Celsius value and can never be negative. A setter would be meaningless.

```
# file: sc_08_17_temperature.py
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius    # uses the setter,
                                # validation happens here

    @property
    def celsius(self):
        return self._celsius
```


8 - Classes and objects - the basics

```
@celsius.setter
def celsius(self, value):
    if value < -273.15:
        raise ValueError(f"Temperature {value} is below
absolute zero")
    self._celsius = value

@property
def fahrenheit(self):
    return self._celsius * 9 / 5 + 32

@fahrenheit.setter
def fahrenheit(self, value):
    self.celsius = (value - 32) * 5 / 9 # uses the celsius
                                       # setter, validation
                                       # for free

@property
def kelvin(self):
    return self._celsius + 273.15 # no setter - read-only

def __str__(self):
    return f"{self.celsius} C | {self.fahrenheit:.1f} F |
{self.kelvin:.2f} K"

if __name__ == "__main__":
    t = Temperature(25)
    print(t)           # 25 C | 77.0 F | 298.15 K
    t.fahrenheit = 32
    print(t.celsius)   # 0.0
    # t.kelvin = 300    # AttributeError: read-only
    # t.celsius = -300  # ValueError: below absolute zero
```

Notice a couple of elegant details. In the constructor we write `self.celsius = celsius`, not `self._celsius = celsius`. This means the setter runs already when the object is created - validation is guaranteed from the very first moment, not only when someone explicitly sets a new value.

The Fahrenheit setter calls `self.celsius = ...` instead of `self._celsius = ...`. This means the Celsius setter's validation runs automatically even when someone sets the temperature via Fahrenheit. We avoid duplicating the validation logic.

The class has one internal value (`_celsius`), but exposes three interfaces externally. The client code chooses whichever scale is most natural - the class handles the rest.

Output

```
25 C | 77.0 F | 298.15 K
0.0
```

8.11 dataclass - when the class is just data

Some classes are simple data containers - they store some values together and may need comparison and readable output, but no significant logic. Let us look at a `Point` class representing a point in a coordinate system:

```
# file: sc_08_15_point_manual.py
class Point:
    def __init__(self, x, y):
        self._x = x
        self._y = y

    def __str__(self):
        return f"Point(x={self._x}, y={self._y})"

    def __eq__(self, other):
        return self._x == other._x and self._y == other._y

p1 = Point(3, 4)
p2 = Point(3, 4)
p3 = Point(1, 2)
print(p1)           # Point(x=3, y=4)
print(p1 == p2)     # True
print(p1 == p3)     # False
```

This is straightforward code, but we write a lot to achieve little. Python has a built-in mechanism for exactly this situation - `@dataclass`:

```
# file: sc_08_15_point_dataclass.py
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int

p1 = Point(3, 4)
p2 = Point(3, 4)
p3 = Point(1, 2)
```

8 - Classes and objects - the basics

```
print(p1)          # Point(x=3, y=4)
print(p1 == p2)    # True
print(p1 == p3)    # False
```

The two versions are functionally equivalent. `@dataclass` is a decorator (we look at decorators more closely in a later chapter) that reads the field declarations and *generates* `__init__()`, `__repr__()` and `__eq__()` *automatically*. We do not have to write them by hand.

Fields are declared with name and type directly in the class block: `x: int` means the class has a field `x` of type `int`. Note that these are plain attributes without a leading underscore - `dataclass` exposes them directly, without properties.

We can also give fields default values:

```
@dataclass
class Point:
    x: int = 0
    y: int = 0

origo = Point()
print(origo)          # Point(x=0, y=0)
```

8.11.1 Default values for mutable fields

The default value syntax we just saw - `x: int = 0` - works for *immutable* values like numbers and strings. But consider a `Bag` class that should hold a list of items, where the default is an empty list:

```
@dataclass
class Bag:
    items: list = []    # ValueError - do not do this
```

Python raises a `ValueError` immediately when the class is defined. The reason is that `[]` is evaluated once, when the class is created - not once per instance. All `Bag` objects would therefore share the exact same list object. Modifying one instance would silently affect all others:

```
# If Python allowed this - which it does not - the result would
# be:
b1 = Bag()
b2 = Bag()
b1.items.append("apple")
print(b2.items)    # ['apple'] - b2 is affected too, which is
wrong
```

This problem is not unique to `@dataclass` - it is a general Python pitfall with mutable default arguments, which we have seen in functions as well. `@dataclass` simply catches it early and refuses, rather than letting the bug appear silently at runtime.

The solution is `field()` with `default_factory`:

```
from dataclasses import dataclass, field

@dataclass
class Bag:
    items: list = field(default_factory=list)

b1 = Bag()
b2 = Bag()
b1.items.append("apple")
print(b1.items)    # ['apple']
print(b2.items)    # [] - separate list, not shared
```

`field()` is a helper function that gives more control over individual fields. It has several parameters - the most important one here is `default_factory`.

The name describes what it does: it is a factory for default values. Instead of giving a ready-made value as the default, we give a function that creates a new value each time. `@dataclass` calls this function automatically when a new instance is created:

```
field(default_factory=list)
# roughly equivalent to:
# for each new object → items = list() → []
```

We pass in `list` not `list()`. We are giving the function itself, not the result of calling it. `@dataclass` calls it for us, once per new instance, so that each object gets its own fresh list.

`default_factory` can take any function with no arguments - typically `list`, `dict` or `set` for empty collections, but also custom functions if more complex default values are needed.

Fact: `field()` and mutable defaults

`default_factory` solves the mutable default problem, but `field()` can do more.

Some useful parameters:

8 - Classes and objects - the basics

`default_factory=list` - call `list()` for each new instance instead of sharing one object

`repr=False` - exclude the field from `__repr__()` output, useful for large or sensitive values

`compare=False` - exclude the field from `__eq__()` comparisons

`init=False` - do not include the field as a constructor parameter; the field gets its value from default or `default_factory` only

A field marked `init=False` with a `default_factory` is useful for computed or cached state that should not be passed in by the caller.

What dataclass gives us - and what it does not

`@dataclass` automatically generates `__init__()`, `__repr__()` and `__eq__()`. With additional arguments we can also get `__lt__()`, `__gt__()` and similar (`@dataclass(order=True)`), and frozen (immutable) objects (`@dataclass(frozen=True)`).

What `@dataclass` does not give us is validation, properties and custom logic. If we want to prevent `x` from being set to a negative number, or compute the distance to the origin as a property, we have to add it ourselves - and then we are essentially back to a regular class. Account from this chapter would not have been simpler with `@dataclass` - the logic requires full control over the constructor and property setters.

Nice to know: type hints in dataclass

`@dataclass` requires type-annotated fields - `x: int` is not optional. This makes `@dataclass` a good argument for using type hints consistently: we get functionality for free as a reward.

8.12 Class methods and class variables

The attributes we have seen so far belong to each individual object. Sometimes we need data that belongs to the class itself - shared between all objects. For Account this could be:

- Next available account number
- Number of accounts created
- Total number of transactions

Class variables are declared inside the class but outside all methods. Python offers two types of class methods:

Class method (@classmethod):

- Used to read or modify class variables
- Receives the class (`cls`) as the first argument, not the object (`self`)
- Annotated with `@classmethod`

Static method (@staticmethod):

- Receives neither `self` nor `cls` - belongs to the class logically, but needs no access to it
- Used for helper functions that need neither object nor class data
- Annotated with `@staticmethod`

Modified code based on `sc_08_06_Account3.py` - only modified or new code is shown here:

```
1: # file: sc_08_08_Account4.py
2: from datetime import datetime
3:
4:
5: class Account:
6:     # Class variable: next available account number
7:     _next_account_no = 1000
8:
9:     # Class variable: number of accounts created
10:    _account_count = 0
11:
12:    # Class variable: total number of transactions performed
13:    _transaction_count = 0
14:
15:    @staticmethod
16:    def is_valid_account_no(account_no):
17:        """Returns True if the account number is an
18:        integer between 1000 and 9999."""
19:        return (isinstance(account_no, int)
20:                and 1000 <= account_no <= 9999)
21:
22:    @classmethod
23:    def set_next_account_no(cls, new_start):
24:        if (new_start > cls._next_account_no
25:            and cls.is_valid_account_no(new_start)):
26:            cls._next_account_no = new_start
27:
```

8 - Classes and objects - the basics

```
28:     @classmethod
29:     def get_account_count(cls):
30:         return cls._account_count
31:
32:     @classmethod
33:     def get_transaction_count(cls):
34:         return cls._transaction_count
35:
36:     def __init__(self, cust_id, start_balance, interest):
37:         self._cust_id = cust_id
38:         self._account_no = Account._next_account_no
39:         Account._next_account_no += 1
40:         Account._account_count += 1
41:         self._balance = start_balance
42:         self._interest = interest
43:         self._transactions = []
44:
45:     # withdraw() and add_monthly_interest()
46:     # not shown, will also update
47:     # Account._transaction_count
48:     def deposit(self, amount):
49:         if amount > 0:
50:             self._balance += amount
51:             self._transactions.append(
52:                 Transaction(amount, "deposit"))
53:             Account._transaction_count += 1
54:         return self._balance
55:
56:
57: # Example usage:
58: if __name__ == "__main__":
59:     # Set new starting point for next available account no
60:     Account.set_next_account_no(2000)
61:
62:     acc1 = Account("A123", 1000, 2.5)
63:     acc2 = Account("B456", 500, 1.8)
64:
65:     acc1.deposit(200)
66:     acc2.withdraw(100)
67:     acc1.add_monthly_interest()
68:
69:     print(acc1)
70:     print()
71:     print(acc2)
72:     print()
73:
74:     print(f"Number of accounts:
{Account.get_account_count()}")
```

```
75:     tx_count = Account.get_transaction_count()
76:     print(f"Number of transactions: {tx_count}")
77:
78:     print()
79:
80:     # Example of using staticmethod:
81:     test_no = 2345
82:
83:     if Account.is_valid_account_no(test_no):
84:         print(f"{test_no} is a valid account number.")
85:     else:
86:         print(f"{test_no} is NOT a valid account number.")
```

Line 6 – 13

Here three class variables are defined. Class variables belong to the class, not any particular object, and are therefore shared by all Account objects.

Line 15-20

`is_valid_account_no()` is marked with `@staticmethod`. This means the method lives in the class but receives neither `self` nor `cls` as a parameter.

The method suits being a static method because it only checks whether an account number is valid. It needs no data from a particular account object or from the class variables.

Line 22-26

`set_next_account_no()` is a class method. It is marked with `@classmethod` and receives the class as the parameter `cls`.

The method uses `cls._next_account_no` to read and possibly change the next available account number. It also uses `cls.is_valid_account_no(new_start)` to check that the new starting number is valid.

Line 28-34

`get_account_count()` and `get_transaction_count()` are class methods because they return values that belong to the class as a whole, not any one particular account.

They can therefore be called directly on the class:

```
Account.get_account_count()
```


8 - Classes and objects - the basics

```
Account.get_transaction_count()
```

Line 36-43

The constructor creates instance variables for the individual account, including customer number, balance, interest rate and transaction list. It also uses the class variables to assign an account number and update the account count.

Line 45-54

`deposit()` is a regular instance method. It uses `self` because the deposit applies to one specific account.

The method updates the balance and transaction list of the object, but also increments the class variable `Account._transaction_count`, since the transaction counter applies to all accounts combined.

The client code under Example usage, read alongside the output below, illustrates the usefulness of class variables and methods.

Output

```
Customer id = A123
Account no  = 2000
Balance     = 1202.50
Interest    = 2.5%

Customer id = B456
Account no  = 2001
Balance     = 400.00
Interest    = 1.8%

Number of accounts: 2
Number of transactions: 3

2345 is a valid account number.
```

8.13 Operator overloading

Python lets us define what it means to use common operators on objects of our own classes. This is called operator overloading and is done by implementing special methods with double underscores - dunder methods.

You have already used this without thinking about it: when you write `"Hello" + " World"` Python calls `str.__add__()`, and when you write `print(object)` Python calls `object.__str__()`.

```
hello_str = "Hello "
```

```
world_str = "World"
total_str = hello_str + world_str    # Python calls implicitly
hello_str.__add__(world_str)
total_str = hello_str.__add__(world_str) # explicit call
```

Here is a Circle class that implements comparison and printing:

```
1: # file: sc_08_09_circle_with_op_overload.py
2: class Circle:
3:     def __init__(self, radius=1):
4:         self._radius = radius
5:
6:     def __str__(self) -> str:
7:         return f"Radius = {self._radius}"
8:
9:     def __eq__(self, other) -> bool:
10:        return self._radius == other._radius
11:
12:    def __lt__(self, other) -> bool:
13:        return self._radius < other._radius
14:
15:    def __gt__(self, other) -> bool:
16:        return self._radius > other._radius
17:
18: def main():
19:     circle1 = Circle()
20:     circle2 = Circle(25)
21:     print(circle1)
22:     print(circle2)
23:     print(f"Are equal? {circle1 == circle2}") # __eq__()
24:     print(f"c1 > c2? {circle1 > circle2}") # __gt__()
25:     print(f"c1 < c2? {circle1 < circle2}") # __lt__()
26:
27: if __name__ == "__main__":
28:     main()
```

Output:

```
Radius = 1
Radius = 25
Are equal? False
c1 > c2? False
c1 < c2? True
```

As the code shows, we can now compare Circle objects with ==, < and > in exactly the same way as we compare numbers and strings.

8.13.1 Overloading indexing and the in operator

Overload the indexing operator

The operator `[]` is used for indexing, slicing and dict lookup. We overload it by implementing `__getitem__()`:

```
# file: sc_08_10_overload_getitem.py
class Order:
    def __init__(self, cart, customer):
        self.cart = list(cart)
        self.customer = customer

    def __getitem__(self, key):
        return self.cart[key]

order = Order(['Soap', 'Apple', 'Deodorant'], 'Hansen')

print(order[0])      # Soap
print(order[-1])     # Deodorant
print(order[0:2])    # ['Soap', 'Apple']
```

When slicing is used, Python generates a slice object and passes it as key to `__getitem__()`. We leave the interpretation to list by writing `self._cart[key]` - list has already implemented `__getitem__()` and handles both regular indices and slice objects.

Overloading the in operator

The `in` operator is used to check whether an element is present in a collection - we have used it with lists, dictionaries and sets. We can support `in` in our own classes by implementing `__contains__()`:

```
# file: sc_08_11_contains.py
class Order:
    def __init__(self, cart, customer):
        self.cart = list(cart)
        self.customer = customer

    def __getitem__(self, key):
        return self.cart[key]

    def __contains__(self, item):
        return item in self.cart

order = Order(['Soap', 'Apple', 'Deodorant'], 'Hansen')
print('Apple' in order)    # True
print('Milk' in order)     # False
```

Without `__contains__()`, Python falls back on iterating through the object using `__iter__()` or `__getitem__()` to check membership - so it would actually work on the `Order` class above even without `__contains__()`, since `__getitem__()` is already defined. Implementing `__contains__()` explicitly is still good practice: it is clearer, and it allows us to use a more efficient lookup if needed - for example checking a set instead of a list.

Overview of common dunder methods:

Operator	Dunder-method
+	<code>__add__()</code>
-	<code>__sub__()</code>
*	<code>__mul__()</code>
/	<code>__truediv__()</code>
==	<code>__eq__()</code>
<	<code>__lt__()</code>
>	<code>__gt__()</code>
[]	<code>__getitem__()</code>
in	<code>__contains__()</code>
<code>str()</code>	<code>__str__()</code>
<code>bool()</code>	<code>__bool__()</code>

Equality, Membership, and Identity

We have now seen three operators that all sit at the same precedence level in Python, but do fundamentally different things:

`==` asks:

do these objects have the same value? - delegates to `__eq__()`, which we can overload

`in` asks:

does this collection contain this element? - delegates to `__contains__()`, which we can overload

`is` asks:

are these the exact same object in memory? - *cannot be overloaded*

The last point is deliberate. Python reserves `is` for itself because identity must always mean identity - there should be no way for a class to redefine what "same object" means. *is is the one operator Python keeps for itself.*

8.14 Overloading vs overriding

These two terms come from object-oriented programming and are easy to mix up.

Overloading means defining multiple versions of the same method with different parameter lists. The compiler picks the right one based on the arguments. This is common in Java and C++:

```
// Java
int add(int a, int b)      { return a + b; }
double add(double a, double b) { return a + b; }
```

Python does not support this directly - if we define two methods with the same name, the second simply replaces the first.

Overriding means replacing a method inherited from a base class with a new version in a subclass. Same name, same signature - just new behaviour.

In Python, all classes implicitly inherit from `object`. When we write `str()`, we override `object.str()` - the version that returns a memory address. When we write `eq()`, we override `object.eq()` - the version that checks identity (same as `is`).

So what Python calls operator overloading is technically a form of overriding: we replace inherited dunder methods to give operators like `==` and `<` new behaviour for our class. The term overloading is used here in a looser, more informal sense - it has become standard vocabulary in Python, even if it does not match the classical definition precisely.

8.15 Truthiness of objects

In chapter 4 we saw that different data types are true or false depending on their value. For our own classes we can define this by implementing `__bool__()`.

```
1: # file: sc_08_12_bool_truthiness.py
2:
3: class Wallet:
```

```
4:     def __init__(self, money=0):
5:         self._money = money
6:
7:     def __bool__(self):
8:         return self._money > 0
9:
10: w1 = Wallet(100)
11: w2 = Wallet(0)
12:
13: print(bool(w1))    # True
14: print(bool(w2))    # False
15:
16: if w1:
17:     print("You have money!")
18: if not w2:
19:     print("Wallet is empty")
20:
21: # Without __bool__(), objects are always truthy
```

Line 6-7

`__bool__()` determines what it means for the object to be true or false. Wallet is true if it has money.

Line 13-14

We can call `__bool__()` explicitly on an object - it calls `__bool__()`. But in practice we rarely do this.

Line 16-19

The typical use: Python calls `__bool__()` implicitly when the object is used in an if-test. `if w1` is more readable than `if w1.has_money()` or `if w1._money > 0`.

Line 21

Classes without `__bool__()` are always truthy - even `Wallet(0)` would be True without lines 7-8. That is Python's default for objects.

Objects without `__bool__()` are always truthy.

Python uses this fallback hierarchy:

1. Does the object have `__bool__()`? Call it.
2. If not: does the object have `__len__()`? True if > 0 , false if 0.
3. Neither: always true (the object exists).

8.16 Review questions

Understanding

1. What is the difference between a class and an object?
2. What does `__init__()` do and when is it called?
3. What is `self`, and why does Python need it?
4. What happens behind the scenes when we write `my_car.drive()`?
5. What does it mean that code is shared by all objects while attributes are unique per object?
6. What is the convention for "private" attributes in Python, and what is the technical reality?
7. What is name mangling, and what was it originally intended for?
8. What is a property, and what is the main advantage of a setter property over direct attribute access?
9. What is a class variable, and how does it differ from an instance variable?
10. What is the difference between `@classmethod` and `@staticmethod`?
11. What is a dunder method, and what happens when we write `circle1 == circle2`?
12. What happens when an object does not implement `__bool__()`?

Practical

13. Create a class `Person` with attributes `name` and `age`, and a method `greet()` that prints a greeting. Create two objects and call the method on both.
14. Add `__str__()` to the `Person` class and print both objects with `print()`.
15. Add a property with validation that prevents `age` from being set to a negative value.
16. Write code that shows that `my_car.drive()` and `Car.drive(my_car)` give the same result.
17. Add `__eq__()` to `BankAccount` so that two accounts are considered equal if they have the same balance.

18. Create a class `Playlist` with a list of songs. Implement `__getitem__()` so that we can use indexing and slicing directly on a `Playlist` object.
19. Implement `__bool__()` in `Playlist` so that an empty playlist is false and one with content is true. Test with `if playlist:`

8.17 Exercises

Rectangle

`ex_08_01_rectangle.py`

Create a class `Rectangle` with the following attributes and methods:

Attributes (use leading underscore):

- `_width`
- `_height`

Methods:

- `area()` - returns the area
- `perimeter()` - returns the perimeter
- `is_square()` - returns True if width equals height
- `__str__()` - e.g. `Rectangle(width=10, height=5)`
- `__eq__()` - two rectangles are equal if they have the same area
- `__lt__()` - a rectangle is less than another if its area is smaller

Extension:

Add a `@staticmethod` `is_valid(width, height)` that returns True if both values are positive. Call it in `__init__()` and raise a `ValueError` if the values are invalid.

Example run:

```
Rectangle(width=10, height=5)
Area:      50
Perimeter: 30
Square?    False
Square?    True
```


8 - Classes and objects - the basics

```
r1 == r2?  False
r1 < r2?   True
r1 > r2?   False
```

Employee

ex_08_02_employee.py

Create a class Employee with the following instance attributes (leading underscore):

- `_name, _employee_id, _department, _salary`

Class variable:

- `_employee_count` - incremented each time an employee is created

Methods:

- `give_raise(amount)` - increase salary by amount. Reject negative amounts.
- `annual_salary()` - returns monthly salary * 12
- `@classmethod get_employee_count()` - returns total employees created
- `__str__()` - formatted output of all attributes
- `__eq__()` - two employees are equal if they have the same `_employee_id`
- `__lt__()` - sort by salary
- `__gt__()` - sort by salary

Example run:

```
Employee ID: 1001
Name:        Alice Johnson
Department:  Engineering
Monthly:     65000
Annual salary: 780000
After raise:  840000
Invalid raise - amount must be positive.

Employees created: 3
```

```
e1 == e2? False  
Highest salary: Clara Lee (72000/month)
```

Fraction

ex_08_03_fraction.py

Create a class Fraction that represents a mathematical fraction.

Attributes:

- `_numerator` - the top number
- `_denominator` - the bottom number (never zero)

Methods:

- `@staticmethod gcd(a, b)` - returns the greatest common divisor using Euclid's algorithm. Used internally to reduce fractions.

- `__init__()` - reduce the fraction automatically on creation
- `__str__()` - returns e.g. 3/4 or 2 (if denominator is 1)
- `__add__()` - add two fractions: $a/b + c/d = (a*d + c*b) / (b*d)$
- `__mul__()` - multiply two fractions: $a/b * c/d = (a*c) / (b*d)$
- `__eq__()` - two fractions are equal if $a*d == b*c$
- `__lt__()` - compare by value

Raise a `ValueError` if denominator is zero.

Example run:

```
Fraction(3, 4):    3/4  
Fraction(6, 8):    3/4    (reduced automatically)  
Fraction(4, 1):    4  
  
1/2 + 1/3  = 5/6  
1/2 * 2/3  = 1/3  
3/4 == 6/8 = True  
1/2 < 3/4  = True  
  
sorted: [1/4, 1/3, 1/2, 2/3, 3/4]
```

8 - Classes and objects - the basics

Vector2D

ex_08_04_vector2d.py

Create a class Vector2D representing a 2D mathematical vector.

Attributes:

- `_x, _y` - the x and y components

Methods:

- `__str__()` - e.g. `Vector2D(3, 4)`
- `__repr__()` - same as `__str__`
- `__add__()` - vector addition: (x_1+x_2, y_1+y_2)
- `__sub__()` - vector subtraction
- `__mul__(scalar)` - scalar multiplication: $(x*s, y*s)$
- `__neg__()` - negate the vector: $(-x, -y)$
- `__abs__()` - magnitude: $\sqrt{x^2 + y^2}$
- `__eq__()` - equal if both components are equal
- `dot(other)` - dot product: $x_1*x_2 + y_1*y_2$

Example run:

```
v1 = Vector2D(3, 4)
v2 = Vector2D(1, 2)

v1:          Vector2D(3, 4)
v2:          Vector2D(1, 2)
v1 + v2:     Vector2D(4, 6)
v1 - v2:     Vector2D(2, 2)
v1 * 3:      Vector2D(9, 12)
-v1:         Vector2D(-3, -4)
|v1|:        5.0
v1.dot(v2):  11
v1 == v2:    False
```

Playlist

ex_08_05_playlist.py

Create a class `Playlist` that manages a music playlist.

Attributes:

- `_name` - playlist name (read-write property)
- `_songs` - internal list of song titles

Methods:

- `add(song)` - add a song
- `remove(song)` - remove a song (print a message if not found)
- `shuffle()` - shuffle the playlist in place (use `random.shuffle`)
- `__len__()` - number of songs
- `__contains__(song)` - in operator support
- `__str__()` - display name and all songs numbered
- `@property name` - getter and setter for the playlist name

Example run:

```
Playlist: Road Trip (4 songs)
 1. Bohemian Rhapsody
 2. Hotel California
 3. Stairway to Heaven
 4. Sweet Child O' Mine

'Hotel California' in playlist: True
'Yesterday' in playlist: False
Length: 4

After removing 'Hotel California':
Playlist: Road Trip (3 songs)
 1. Bohemian Rhapsody
 2. Stairway to Heaven
 3. Sweet Child O' Mine
```

Card Deck

`ex_08_06_card_deck.py`

Create two classes: `Card` and `Deck`.

8 - Classes and objects - the basics

Card

Attributes:

- `_suit`
- `_value`

Methods:

- `__str__()` - e.g. Ace of Spades, 10 of Hearts
- `__repr__()` - same as `__str__`

Deck

Represents a standard 52-card deck.

Methods:

- `__init__()` - create all 52 cards automatically
- `shuffle()` - shuffle the deck using `random.shuffle`
- `draw()` - remove and return the top card. Raise `IndexError` if empty.
- `__len__()` - number of remaining cards
- `__getitem__(index)` - support indexing: `deck[0]`
- `__str__()` - show how many cards remain

Use these suits and values:

```
SUITS = ["Clubs", "Diamonds", "Hearts", "Spades"]
VALUES = ["2", "3", "4", "5", "6", "7", "8", "9", "10",
          "Jack", "Queen", "King", "Ace"]
```

Example run:

```
New deck: 52 cards
Top card: 2 of Clubs
deck[0]: 2 of Clubs
```

After shuffle - top 5 cards:

```
King of Hearts
7 of Diamonds
3 of Spades
Ace of Clubs
10 of Hearts
```

```
Drawing 3 cards: King of Hearts, 7 of Diamonds, 3 of Spades
```

Cards remaining: 49

Student Register

ex_08_07_student_register.py

Create two classes: Student and StudentRegister.

Student

Attributes:

- `_name`, `_student_id`, `_gpa` (grade point average, 0.0-4.0)

Class variable: `_student_count`

Methods:

- `@classmethod get_count()` - total students created
- `@property gpa` - getter and setter (validate 0.0-4.0)
- `__str__()` - formatted output
- `__lt__()` - compare by GPA (for sorting)

StudentRegister

Methods:

- `add(student)` - add a student
- `remove_by_id(student_id)` - remove student with given ID
- `find_by_name(name)` - return matching students (case-insensitive partial match)
- `top_students(n)` - return the n students with highest GPA
- `__len__()` - number of students
- `__str__()` - display all students